

1. 加锁协议的基本思想是什么?它如何确保正确执行并发事务?采用锁定协议后必须解决的问题是什么?

2. 传统数据管理经历了人工管理、文件系统、数据库系统等阶段,在这一过程中,数据独立性越来越高,为什么数据管理系统的数据独立性越高越好?(阐明数据的两种独立性的含义和意义)

3. 加锁是现代数据库系统中事务并发管理的重要技术,试回答下述问题:

(1) 已有的 (S, X)、(S, U, X) 锁能解决事务并发中的死锁问题么?为什么?

(2) (S, U, X) 锁的相容矩阵如下图,为什么已经加了 U 锁,不允许其它事务申请加 U 锁?如果允许会出现什么情况?

其它事务已拥有的锁				
锁 请 求		S	U	X
	S	Y	Y	N
	U	Y	N	N
	X	N	N	N

4. 假设运行记录与数据库的存储磁盘有独立失效模式,介质失效恢复时,对运行记录中上一检查点以前的已提交事务应该 redo 否?为什么?什么时候可以清空 CTL 表?

5. 某网上商品销售系统的业务流程如下:

(1) 将客户的订单记录(订单号, 客户 ID, 商品 ID, 购买数量)写入订单表;

(2) 将库存表(商品 ID, 库存量)中订购商品的库存量减去该商品的购买数量。

针对上述业务流程,引入如下伪指令:

- 将商品 A 的订单记录插入订单表记为 $I(A)$;
- 读取商品 A 的库存量到变量 x , 记为 $x = R(A)$;
- 变量 x 值写入商品 A 中的库存量, 记为 $W(A, x)$ 。
- 则客户 i 的销售业务伪指令序列为: $I_i(A)$, $x_i = R_i(A)$, $x_i = x_i - a_i$, $W_i(A, x_i)$. 其中 a_i 为商品的购买数量。

假设当前库存量足够,不考虑发生修改后库存量小于 0 的情况。若客户 1、客户 2 同时购买同一种商品时,可能出现的执行序列为:

$I_1(A)$, $I_2(A)$, $x_1 = R_1(A)$, $x_2 = R_2(A)$, $x_1 = x_1 - a_1$, $W_1(A, x_1)$, $x_2 = x_2 - a_2$, $W_2(A, x_2)$ 。

请回答以下问题:

(1) 客户此时会出现什么问题?

(2) 为了解决上述问题,引入共享锁指令 $S\text{Lock}(A)$ 和独占锁指令 $X\text{Lock}(A)$ 对数据 A 进行加锁,解锁指令 $\text{Unlock}(A)$ 对数据 A 进行解锁,客户 i 的加锁指令用 $S\text{Lock}_i(A)$ 表示,其他类同。插入订单表的操作不需要引入锁指令。请补充上述执行序列,使其满足 2PL 协议,并使持有锁的时间最短。

6. 设数据库中包含下列关系:

course(cno, cname, dame)/*课程号, 课程名, 开课系名*/

enroll(sid, grade, cno, dname1, sectno, pname, dname2)/*学号, 成绩, 课程号, 开课系名, 分班号, 任课教师名, 任课教师所在系名*/

student(sid, sname, sex, age, year, gpa)/*学号, 姓名, 性别, 年龄, 年级, 成绩点*/

试编写一个触发器, 监视上面数据库中 enroll 表上的 INSERT 操作, 对每条 INSERT 语句, 判断其插入元组的 grade 值是否小于 3.0, 将这样的元组自动插入到 failedcourse 表中(设 failedcourse 表与 enroll 表的模式完全相同)。